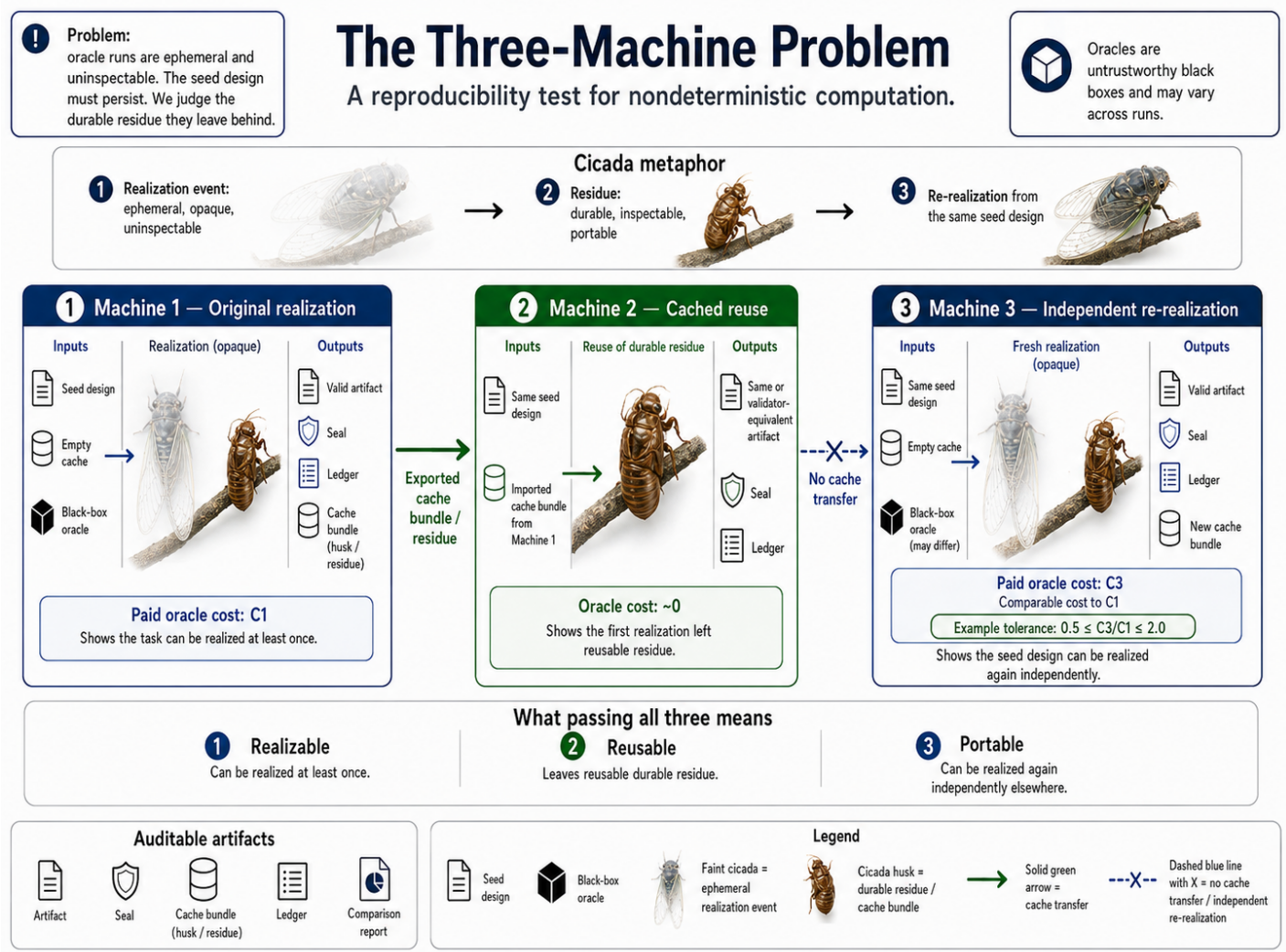


The Three-Machine Problem

A reproducibility test for nondeterministic computation

Summary. Modern AI systems often present a successful run as evidence of capability. A model receives a task, calls tools, writes files, and the demonstration ends. That standard is too weak for engineering. A successful run is an event. Durable infrastructure requires “residue” that can be checked, moved, reused, and independently realized again. The three-machine problem is a compact test for that standard. It asks whether a seed design can produce a valid artifact on one machine, whether the resulting cache can collapse repeated cost on a second machine, and whether the same seed can be realized again, on a third machine at comparable cost without sharing the cache.



1. Working terms

- **Seed design.** A portable build specification. It describes the intended artifact, required inputs, allowed oracle use, declared outputs, validators, and cost constraints. The seed design describes the structure from which an artifact of the intended specification may be realized.
- **Oracle.** A nondeterministic process used during a build. In current systems this is often a language model, but the term is broader. An oracle may generate code, plans, tests, explanations, or intermediate materials. It is not trusted by default. Its output must be checked.
- **Realization.** The act of converting a seed design into concrete artifacts through execution. A realization may involve deterministic steps, oracle calls, cache lookup, and validators. It is an event, with an opaque process.
- **Residue.** The durable material left behind by a realization event. Residue may include output files, seals, traces, cost records, validators, and cache entries. Residue is what can be inspected, reused, transferred, or replaced by deterministic code.
- **Seal.** A structured or cryptographic binding between an artifact and the conditions that produced it. A seal records enough information to check whether an output matches its declared recipe, inputs, validators, and build semantics.
- **Validator.** A deterministic check that decides whether an artifact satisfies the seed design. Validators define the operational meaning of success. They do not prove universal correctness, instead define bounded acceptance.
- **Reuse.** The use of sealed residue from a previous realization to avoid paying for the same oracle-mediated work again. In the three-machine problem, Machine 2 tests reuse.
- **Re-realization.** An independent build from the same seed design without using the prior cache. In the three-machine problem, Machine 3 tests whether the seed design carries enough structure to produce a valid artifact again at comparable cost.
- **Comparable cost.** A bounded relationship between two independent realizations. The requirement is not exact equality. Machine 3 should realize a valid artifact within a declared tolerance of Machine 1's oracle calls, token cost, wall time, or other measured energy budget.
- **Artifact equivalence.** A declared relationship between outputs from different machines. Equivalence may be bitwise identity, matching content hashes, matching seals, or validator-bounded acceptance. The seed design must state which form is required.
- These terms avoid treating the oracle as an authority. The system judges only what the event leaves behind: artifacts, seals, traces, validators, and cost records.

2. The problem

Nondeterministic computation creates an audit problem. The internal process of a model call is not available as a stable engineering object. A user can inspect prompts, tool calls, emitted files, logs, validation results, and costs. The hidden computation that produced them is not directly available for inspection, replay, or certification.

This makes ordinary demonstrations fragile:

A transcript may show that an agent produced an answer. It does not show whether the work is reusable. It does not show whether the cost was recorded. It does not show whether the output can be verified elsewhere. It does not show whether another clean machine can realize the same design under similar bounds.

A practical standard should avoid claims about hidden cognition. It should judge the event by what survives it. The relevant questions are concrete: what was produced, what did it cost, what was validated, what can be reused, and what can be realized again elsewhere.

3. The three-machine test

The three-machine problem separates three claims that are often treated as one.

1. **Original realization.** Machine 1 receives a seed design, an empty cache, and oracle access. It builds a valid artifact and reports paid oracle cost C_1 .
2. **Cached reuse.** Machine 2 receives the same seed design and a cache exported from Machine 1. It produces the same or validator-equivalent artifact with zero or near-zero oracle cost.
3. **Independent re-realization.** Machine 3 receives only the same seed design, an empty cache, and oracle access. It independently builds a valid artifact and reports cost C_3 , where C_3 is comparable to C_1 under a declared tolerance.

Machine 1 proves that the task can be realized at least once. Machine 2 proves that the first realization left reusable residue. Machine 3 proves that the seed design is portable enough to realize valid work without the original cache. These are different properties. Cached reuse is deterministic materialization from sealed residue. Independent re-realization is bounded repetition of a nondeterministic process under the same build contract.

A system that only passes Machine 1 has shown a demo. A system that passes Machine 1 and Machine 2 has shown that the demo produced valid, reusable and sealed artifacts. A system that passes all three has shown that the seed design travels, that cost can be accounted for, and that the audited artifacts can enter a durable workflow.

4. Required artifacts

A minimal implementation needs only the artifacts required to make the test auditable.

- **Seed design:** a portable build specification, including rules, declared inputs, declared outputs, validators, oracle bounds, and cost budget.
- **Transaction record:** evidence that each rule executed in quarantine and committed outputs only after validation.
- **Seal:** a binding between output content, recipe identity, input signatures, rule identity, and build semantics.
- **Cache bundle:** portable realized residue from Machine 1, including sealed outputs and metadata needed to validate cache hits on Machine 2.
- **Ledger:** a machine-readable account of oracle calls, paid cost, reuse events, fuel use, validation status, and artifact roots.
- **Comparison report:** a final report comparing the three machines on validity, output equivalence, oracle calls, reuse, and cost.

The seed design and cache bundle must remain distinct. Machine 2 receives both. Machine 3 receives only the seed design. This distinction prevents cache reuse from being mistaken for reproducibility, and it prevents independent re-realization from being mistaken for deterministic identity. The design is identified as a portable constraint that lets another machine produce a valid artifact under the same acceptance conditions.

5. Measurement and development implication

The test should report a small set of facts: whether each machine produced a valid artifact, whether Machine 2 used the imported cache rather than the oracle, whether Machine 2 produced the same artifact root or passed the declared equivalence relation, whether Machine 3 built from the seed design with an empty cache, whether C_3 was comparable to C_1 , and whether all cost and reuse decisions were recorded in one ledger format.

The tolerance for comparable cost must be explicit. For live oracle runs, it may be expressed as a ratio, for example $0.5 \leq C_3/C_1 \leq 2.0$, or as a bounded difference in calls and token cost. For stub tests, synthetic cost and oracle-call count are acceptable, but the report shape should match the live path.

Equivalence must also be explicit. The test should not claim semantic sameness in the abstract. Two artifacts are equivalent only relative to declared outputs, seals, validators, and acceptance conditions. This keeps the claim operational and auditable.

The standard also creates pressure to reduce oracle dependence over time. An oracle is useful when deterministic implementation is missing. Once it produces stable residue, that residue can be sealed, reused, inspected, and gradually replaced by deterministic code. Nondeterminism is scaffolding. Determinism is the product.

The claim should be narrow and falsifiable:

Nondeterministic oracle calls can be converted into portable, verifiable, cost-accounted build artifacts for a seed design, while preserving comparable-cost realization from the same seed design using fresh oracle calls on an independent machine.

This is stronger than a successful demo and weaker than a claim of full bit-by-bit reproducibility. It sets a useful beta standard for systems that use live oracles during build execution. It asks a system to leave evidence: artifact, seal, cache, ledger, and a second clean realization.